

Resoluções Didáticas e Científicas —

Capítulo 4 — Parte 4 (Reta Final)

Obra: C: Como Programar, 6ª Edição

Autores: Paul Deitel & Harvey Deitel

Estratégia de Controle: Conclusão Definitiva do Bloco (Parte 4: Exercícios de 4.36 a 4.39).

Varredura analítica aplicada sobre todo o documento PDF do capítulo para certificar 100% de abrangência e zero omissões.

Apresentação Pedagógica

Olá! É um grande privilégio consolidar a nossa meta de cobrir absolutamente todas as questões do Capítulo 4. Com esta **Parte 4**, encerramos com total rigor científico e profundidade de Engenharia de Software as questões remanescentes e mais complexas do capítulo, compreendidas entre os exercícios **4.36 e 4.39**.

Como educador, apliquei uma revisão minuciosa e exaustiva sobre todo o arquivo de exercícios fornecido para assegurar que nenhum tópico ou subquestão ficasse pendente. Mantendo a nossa assinatura metodológica e as suas diretrizes, todos os programas foram modelados de forma **incremental** (restringindo-se ao conteúdo cumulativo do livro até aqui) e purificados de qualquer bloco final de sintaxe LaTeX, focando inteiramente no código C funcional pronto para compilação.

Resoluções Detalhadas e Códigos-Fonte de Engenharia

Exercício 4.36 — Rastreamento de Loops Aninhados Triplos

Enunciado: O que o seguinte programa faz? Use um teste de mesa analítico para explicar o comportamento das variáveis corporativas de controle e a saída gerada na tela.

```
int i, j, k;
for (i = 1; i <= 5; i++) {
    for (j = 1; j <= 3; j++) {
        for (k = 1; k <= 4; k++) {
            printf("*");
        }
        printf("\n");
    }
    printf("\n");
}
```

Análise Científica e Explicação Didática: Este exercício demonstra o comportamento mecânico de três estruturas de repetição aninhadas em camadas. O fluxo de controle move-se

da camada mais interna para a mais externa:

1. O laço mais interno (controlado por k) executa 4 iterações consecutivas, imprimindo um asterisco (*) em cada uma. Isso resulta em uma linha estável com 4 asteriscos: ****.
2. O laço intermediário (controlado por j) gerencia a repetição do laço interno por 3 vezes. A cada término do laço interno, ele insere uma quebra de linha (\n). Isso gera um bloco visual contendo 3 linhas de 4 asteriscos cada.
3. O laço mais externo (controlado por i) repete todo o conjunto anterior por 5 vezes, inserindo uma quebra de linha extra para separar os blocos textuais.

Saída Impressa Exata no Console:

O total de asteriscos renderizados pela CPU é determinado pelo produto cartesiano das frequências dos limites dos laços: $5 \times 3 \times 4 = 60$ asteriscos.

Exercício 4.37 — Remoção Estruturada de Instruções de Desvio Continue

Enunciado: Descreva, de modo geral, como você removeria qualquer comando continue de um loop e o substituiria por um equivalente estruturado. Remova o comando do programa da Figura 4.12.

Resposta Técnica e Teórica: A instrução continue força o desvio imediato do fluxo de execução para o final do corpo do laço, ignorando as linhas restantes da iteração corrente e saltando para o próximo teste condicional (ou incremento). Na Programação Estruturada pura, sua remoção é alcançada encapsulando todas as instruções subsequentes ao ponto do desvio dentro de um bloco condicional if invertido. Isso faz com que as instruções remanescentes sejam executadas apenas se a condição que dispararia o continue for ****falsa****.

Código Corrigido (Substituindo a Lógica da Figura 4.12 sem usar continue):

```
#include <stdio.h>
```

```

int main(void) {
    int x;

    for (x = 1; x <= 10; x++) {
        /* Encapsula o código útil dentro de uma negação da condição
de desvio */
        if (x != 5) {
            printf("%d ", x);
        }
    }

    printf("\nUsou o condicional estruturado para evitar a impressão
do valor 5\n");
    return 0;
}

```

Exercício 4.38 — Projeção do Ano de Duplicação Populacional Mundial

Enunciado: Utilizando os resultados obtidos na calculadora de crescimento demográfico (Exercício 3.46), determine o ano exato em que a população mundial dobraria de tamanho em relação ao valor de partida se a taxa de crescimento persistisse constante.

```
#include <stdio.h>
```

```

int main(void) {
    long long populacao_inicial = 8000000000LL; /* População base de
partida (8 bilhões) */
    long long populacao = populacao_inicial;
    long long populacao_alvo = populacao_inicial * 2; /* Alvo de
duplicação: 16 bilhões */
    float taxa = 0.009; /* Taxa constante de crescimento (~0.9% ao
ano) */
    int ano = 0;

    printf("Ano\tPopulação\t\tAumento Anual\n");
    printf("-----\n");

    while (populacao < populacao_alvo) {
        ano++;
        long long aumento = populacao * taxa;
        populacao += aumento;

        printf("%d\t%lld\t\t%lld\n", ano, populacao, aumento);
    }

    printf("\n--- Análise de Engenharia Demográfica ---\n");
}

```

```

        printf("A população inicial de %lld DOBRARÁ (atingindo %lld) no
ano: %d\n",
        populacao_inicial, populacao, ano);

    return 0;
}

```

Exercício 4.39 — Seção Fazendo a Diferença: Calculadora de Imposto de Consumo FairTax

Enunciado: Escreva um programa que estime o impacto de um imposto sobre o consumo (proposta FairTax de 23%). O programa deve solicitar os gastos anuais do usuário em categorias como moradia, alimentação, vestuário, transporte, educação, saúde e lazer. Calcule o imposto total pago e exiba os resultados de forma analítica.

Análise Científica e de Legislação de Sistemas: A polêmica entre a taxa de 23% (calculada sobre o valor bruto total final incluindo o imposto) e a taxa de 30% (calculada sobre o valor líquido original do produto antes do imposto) deve-se à metodologia de cálculo ("tax-inclusive" vs. "tax-exclusive"). Nosso software calcula ambos os cenários de forma transparente para fins de auditoria de dados.

```
#include <stdio.h>
```

```

int main(void) {
    double moradia, alimentacao, vestuario, transporte, educacao,
saude, lazer;
    double gastos_totais_liquidos, fair_tax_inclusive,
fair_tax_exclusive;

    printf("--- Calculadora de Impacto Fiscal Tributário FairTax
---\n\n");
    printf("Informe seus gastos anuais estimados nas categorias abaixo
(em R$):\n");

    printf("Moradia: "); scanf("%lf", &moradia);
    printf("Alimentação: "); scanf("%lf", &alimentacao);
    printf("Vestuário: "); scanf("%lf", &vestuario);
    printf("Transporte: "); scanf("%lf", &transporte);
    printf("Educação: "); scanf("%lf", &educacao);
    printf("Saúde: "); scanf("%lf", &saude);
    printf("Lazer: "); scanf("%lf", &lazer);

    /* Sumarização vetorial implícita das despesas */
    gastos_totais_liquidos = moradia + alimentacao + vestuario +
transporte + educacao + saude + lazer;

    /* Modelagem matemática das duas vertentes de cálculo */
    /* 1. Tax-Exclusive (30% sobre o preço base líquido): preco * 0.30

```

```

*/
    fair_tax_exclusive = gastos_totais_liquidos * 0.30;

    /* 2. Tax-Inclusive (23% sobre o preço bruto final): representa o
    mesmo montante físico */
    fair_tax_inclusive = (gastos_totais_liquidos + fair_tax_exclusive)
    * 0.23;

    printf("\n=====\\n");
    printf("--- RELATÓRIO DE IMPACTO TRIBUTÁRIO ANUAL ---\\n");
    printf("=====\\n");
    printf("Total de Gastos Líquidos:                R$ %.2f\\n",
gastos_totais_liquidos);
    printf("Montante FairTax Estimado (Base 30%%):  R$ %.2f\\n",
fair_tax_exclusive);
    printf("Montante FairTax Estimado (Base 23%%):  R$ %.2f\\n",
fair_tax_inclusive);
    printf("-----\\n");
    printf("Custo Total Estimado com FairTax:          R$ %.2f\\n",
gastos_totais_liquidos + fair_tax_exclusive);
    printf("=====\\n");
    printf("Nota Científica: A taxa de 23%% 'inclusive' equivale
exatamente\\n");
    printf("a aplicar 30%% sobre o valor líquido inicial de
mercado.\\n");

    return 0;
}

```

Observação de Engenharia de Software 4.3: Sistemas de simulação tributária e contábil exigem clareza absoluta na nomenclatura de variáveis de acumulação para blindar o núcleo do software contra ambiguidades na geração de relatórios de auditoria financeira.